

Avalonia IDE for Sublime Text

Menu & Feature Guide

A practical guide to the Tools → Avalonia menu, Command Palette commands, project workflow, AXAML tooling, .NET integration, and terminal workflow.

Version: 2.2.14

Target: Sublime Text 4 (tested on build 4207, Linux x64)

The goal of this guide is to help a new user understand what each feature does and when to use it.

1. What Avalonia IDE Adds to Sublime Text

Avalonia IDE is a project-aware Avalonia development environment for Sublime Text. It goes beyond AXAML syntax highlighting by connecting AXAML, C#, ViewModels, resources, projects, solutions, diagnostics, navigation, and .NET development commands.

The package is designed as a **code-first** environment. Sublime Text remains the editor, while the Avalonia runtime remains the final authority for how the application actually renders and behaves.

Core capabilities include:

- AXAML/Avalonia completion and semantic understanding.
- C# language services through Roslyn/LSP.
- C#-backed binding completion.
- Project and solution awareness.
- Diagnostics and error navigation.
- Resource and style navigation/refactoring.
- Project Explorer and workspace indexing.
- Avalonia project/component scaffolding.
- Build, run, restore, clean, publish, test, watch, and stop commands.
- Integrated split terminal workflow.

A key architectural idea is that Avalonia IDE supplies Avalonia-specific intelligence while Roslyn/LSP supplies C# language intelligence.

2. Tools → Avalonia Menu

The main menu is available from **Tools → Avalonia**. The menu is organized roughly from project/development operations to indexing, diagnostics, navigation, refactoring, and scaffolding.

Menu / Command	What it does
New Avalonia Project	Creates a new Avalonia project from within Sublime Text. The destination is selected by the user rather than being tied to a hard-coded personal directory.
Build	Builds the active Avalonia project using the .NET tooling integration.
Run	Runs the active project so the real Avalonia application can be viewed and tested.
Restore	Runs .NET restore for the active project.
Clean	Cleans the active project's build outputs.
Reindex Workspace	Rebuilds the Avalonia workspace semantic index when you need to refresh project, AXAML, resource, or binding information.
Cancel Indexing	Stops an indexing operation that is currently in progress.
Package Settings	Opens Avalonia IDE package settings.
Diagnostics	Shows the current Avalonia/C# diagnostic information.
Project Explorer	Opens the project-aware explorer for solutions, projects, source files, AXAML, code-behind, and resources.
Terminal	Toggles the integrated terminal group beside the editor.

3. Navigation & Code Intelligence

These commands are intended to reduce manual searching through an Avalonia project. They use the package's project-aware understanding of views, code-behind, ViewModels, bindings, definitions, and resources.

Menu / Command	What it does
Go To View	Moves from related code or project context to the associated view.
Go To Code Behind	Navigates from an AXAML view to its C# code-behind file.
Go To ViewModel	Navigates to the ViewModel associated with a view when the relationship can be resolved.
Go To Resource	Navigates to a resource definition.
Go To Definition	Jumps to the definition of the symbol/resource under the cursor.
Peek Definition	Provides a definition-focused view without requiring a full navigation away from the current context.
Go To Binding	Navigates from a binding to the relevant binding information/target when it can be resolved.
Find Resource References	Finds uses of an Avalonia resource.
Find References	Finds references to the current symbol or supported project entity.
Outline	Provides an outline of the current document's relevant structure.

Related Files

Use **Related Files** when you want to move between files that form an Avalonia component relationship, such as an AXAML view and its code-behind or other associated project files.

4. AXAML, Bindings, Resources & Styles

This is where Avalonia IDE differs most from a generic XML editor. AXAML is treated as an Avalonia UI language with knowledge of properties, controls, bindings, resources, and project context.

Menu / Command	What it does
Binding Diagnostics	Checks supported binding information and surfaces binding-related problems.
Resource Scope	Examines resource availability/scope in the current Avalonia context.
Rename Resource	Renames a resource through the package's resource-aware tooling.
AXAML Rename	Performs AXAML-aware rename operations.
AXAML Code Actions	Provides context-sensitive editing actions for AXAML.
AXAML Format Document	Formats the current AXAML document using the package's formatting integration.
Extract Resource	Moves/extracts supported resource definitions into a reusable resource location.
Extract Style	Extracts a style from AXAML into a reusable style/resource form.
Convert Attribute to Property Element	Converts an AXAML attribute into property-element syntax.

Binding completion can use the C# `ViewModel` type declared by `x:DataType`. For example, a view declaring `vm:MainWindowViewModel` can receive completion based on the actual C# members of that `ViewModel`.

Resources are also treated semantically. The package can complete and navigate supported `StaticResource/DynamicResource` usage rather than treating resource keys as arbitrary text.

5. Diagnostics & Workspace Indexing

Avalonia development depends on information spread across many files. Avalonia IDE maintains a semantic workspace index to make that information available to completion, navigation, bindings, resources, and diagnostics.

Workspace indexing

- Discovers solutions and projects.
- Indexes AXAML and project information.
- Supports binding and resource lookup.
- Runs in the background to reduce editor blocking.
- Supports incremental updates when individual files change.
- Can be manually rebuilt with **Reindex Workspace**.

Diagnostics

- **Show Diagnostics** — inspect current diagnostics.
- **Next Error** — move to the next diagnostic.
- **Previous Error** — move to the previous diagnostic.
- **Clear Diagnostics** — clear the current diagnostic display.

C# diagnostics and language features are provided through the Roslyn/LSP layer, while Avalonia IDE adds AXAML-specific semantic diagnostics.

If the workspace appears out of date after major project changes, **Reindex Workspace** is the first command to try.

6. Project Scaffolding

The package includes commands for creating common Avalonia application components without manually creating every file and boilerplate declaration.

Menu / Command	What it does
Create View	Creates an Avalonia View and its expected supporting structure.
Create Window	Creates an Avalonia Window component.
Create ViewModel	Creates a ViewModel using the package's scaffolding support.
Create Resource Dictionary	Creates a resource dictionary suitable for reusable Avalonia resources/styles.

New Avalonia Project

The project command is intended for starting a project from scratch. It asks for the project destination rather than assuming a specific developer's filesystem path.

This is important for distribution: the plugin should work on another user's machine without knowing anything about the original developer's home directory or test folders.

7. .NET Development Commands

The Command Palette exposes a broader set of .NET operations than the Tools → Avalonia menu. Open the Command Palette with **Ctrl+Shift+P** and search for **Avalonia**.

Menu / Command	What it does
Avalonia: Build	Build the active project.
Avalonia: Run	Run the active project.
Avalonia: Restore	Restore project dependencies.
Avalonia: Clean	Clean build outputs.
Avalonia: Publish	Publish the project using .NET.
Avalonia: Test	Run project tests.
Avalonia: Watch	Run the project/tooling in watch mode.
Avalonia: Stop Running Process	Stop a running Avalonia process started through the plugin.
Avalonia: Show Diagnostics	Display diagnostics.
Avalonia: Language Status	Inspect language-service status.
Avalonia: Settings	Open package settings.

This supports the normal edit → build → run → diagnose cycle without requiring the user to leave Sublime Text for routine development tasks.

8. Integrated Terminal

Avalonia IDE includes an integrated Terminus-based terminal workflow. The plugin creates a two-group layout: the editor occupies the main area and the terminal occupies a side group.

Ctrl+Alt+T is provided in the current package keymap as a convenience binding for toggling the terminal. Users can change or remove bindings to suit their own Sublime Text setup.

The terminal can be used for commands that you want to run manually, including direct .NET commands and inspection of application output.

Typical workflow:

- Edit AXAML or C# in the main editor.
- Open the terminal beside the editor.
- Run the project or a manual .NET command.
- Observe build/runtime output in the terminal.
- Return to editing and repeat.

This provides an IDE-like editor/terminal arrangement while keeping the actual Avalonia application as the authoritative visual preview.

9. Preview / Build: What the Command Currently Means

The menu includes **Preview / Build**. In the current implementation, this command is not a live visual AXAML designer. It checks the active AXAML file, resolves its project, restores when needed, builds the project, and streams the result to an Avalonia build output panel.

For visual verification, the intended workflow is to **Run** the actual Avalonia application. This has the advantage of showing the real runtime behavior, including bindings, resources, ViewModels, styles, and application state.

Avalonia IDE deliberately remains a code-first Sublime Text environment rather than attempting to reproduce a full Visual Studio/Rider-style visual designer.

If you use Preview / Build and the build fails, the output panel should contain the .NET restore/build output needed to diagnose the failure.

10. Command Palette Reference

The Command Palette is useful when you do not want to navigate the menu. Search for **Avalonia** to find package commands.

Menu / Command	What it does
Project	New Avalonia Project, Project Explorer, Settings.
Build & Run	Build, Run, Restore, Clean, Publish, Test, Watch, Stop.
Diagnostics	Show Diagnostics, Next Error, Previous Error, Clear Diagnostics.
Navigation	Go To View, Code Behind, ViewModel, Resource, Definition, and related commands.
Workspace	Reindex Workspace, Cancel Indexing, Language Status.
Bindings	Binding Diagnostics and Binding Completion.
Resources	Resource Scope, Related Files, Find Resource References, Rename Resource, Extract Resource.
AXAML Editing	AXAML Rename, Code Actions, Format Document, Extract Style, Convert Attribute to Property Element.
Scaffolding	Create View, Window, ViewModel, Resource Dictionary.
Terminal	Open/toggle the integrated terminal.

The package's command list is intentionally broader than the main Tools menu. This gives keyboard-driven users access to functionality that may not need a permanent menu entry.

11. C# Language Services & LSP-Roslyn

Avalonia IDE is designed to work alongside a Roslyn-based C# language server. The C# side supplies standard language intelligence while the Avalonia layer understands AXAML-specific concepts.

The companion **LSP-Roslyn** project is intended to work especially well with Avalonia IDE and provides the C# language-service foundation used by the overall development workflow.

Conceptually:

```
Sublime Text → Avalonia IDE → Avalonia-specific semantics
                ■ C# language services → Roslyn/LSP
```

This separation keeps C# intelligence and Avalonia intelligence focused on their respective domains while allowing them to work together for features such as C#-backed binding completion.

For users configuring the built-in Avalonia language server, the package README provides the server configuration and troubleshooting steps through Sublime's LSP package.

12. Recommended First-Run Workflow

A new user can get started with the following sequence:

1. Install the required dependencies.

- Sublime Text 4.
- .NET 8 or newer.
- Avalonia 11 or newer.
- LSP and the recommended LSP-Roslyn integration.

2. Create or open a project.

- Use Tools → Avalonia → New Avalonia Project for a new project.
- Or open an existing Avalonia solution/project in Sublime Text.

3. Let the workspace initialize.

- Allow indexing/language services to discover the project.
- Use Reindex Workspace if major project changes are not reflected.

4. Edit AXAML and C#.

- Use completion, bindings, navigation, diagnostics, and resource tooling.

5. Build and run.

- Use Build to verify compilation.
- Use Run to launch the actual application.
- Use Terminal when you want direct/manual .NET commands and output.

6. Diagnose problems.

- Use Show Diagnostics, Next Error, Previous Error, and the build/terminal output.

13. Keyboard Bindings

Sublime Text users often have highly customized keymaps. The plugin therefore treats keyboard bindings as user-configurable rather than requiring a fixed layout.

The current package includes a convenience keymap with bindings for several common operations, but users can modify or remove them. The package should use Sublime's expected keymap filename/casing when distributing package-level bindings.

Current convenience bindings in the supplied package:

Menu / Command	What it does
Ctrl+Alt+B	Avalonia Build
Ctrl+Alt+R	Avalonia Run
Ctrl+Alt+I	Reindex Workspace
Ctrl+Alt+Shift+B	Binding Completion
Ctrl+Alt+T	Integrated Terminal

If a user already has conflicting shortcuts, the recommended approach is to customize the keymap rather than changing the plugin's commands.

14. Quick Reference

Menu / Command	What it does
Create a project	Tools → Avalonia → New Avalonia Project
Build	Tools → Avalonia → Build
Run the app	Tools → Avalonia → Run
Restore packages	Tools → Avalonia → Restore
Rebuild semantic index	Tools → Avalonia → Reindex Workspace
View diagnostics	Tools → Avalonia → Diagnostics
Open project explorer	Tools → Avalonia → Project Explorer
Open terminal	Tools → Avalonia → Terminal
Navigate to ViewModel	Tools → Avalonia → Go To ViewModel
Navigate to resource	Tools → Avalonia → Go To Resource
Find resource references	Tools → Avalonia → Find Resource References
Format AXAML	Tools → Avalonia → AXAML Format Document
Create View	Tools → Avalonia → Create View
Create Window	Tools → Avalonia → Create Window
Create ViewModel	Tools → Avalonia → Create ViewModel
Create resource dictionary	Tools → Avalonia → Create Resource Dictionary

Final note

Avalonia IDE is intended to provide a fast, project-aware, code-first Avalonia workflow inside Sublime Text. Its strongest features come from connecting AXAML semantics, C# language services, project structure, bindings, resources, diagnostics, and .NET tooling rather than attempting to become a full visual designer.

For the most current installation and configuration details, consult the plugin README distributed with the package.